

2. Отличие строгого включения от нестрогого заключается в том, что при строгом множества не должны равняться(то есть их размеры не должны совпадать, что и присутствует в моей программе), а в нестрогом их размеры могут быть одинаковые

3. Ответ в задании 1) {5, 6, 1, 3, 8}

```
#include <stdio.h>

void inputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
}

void outputArray(const int *const a, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
}

void insertionSort(int *a, const size_t size) {
    for (size_t i = 1; i < size; i++) {
        int t = a[i];
        int j = i;
        while (j > 0 && a[j - 1] > t) {
            a[j] = a[j - 1];
            j--;
        }
        a[j] = t;
    }
}

void unionSets(int const *a, int sizeA, int const *b, int sizeB, int *c, int
*sizeC) {
    int n = 0;
    //записываем все числа из массива a в массив c
    for (int i = 0; i < sizeA; i++) {
        c[n] = a[i];
        n++;
    }

    //если элемент из массива b не совпадает с элементом из массива a, к
счётчику добавляем 1
    int count = 0;
    for (int j = 0; j < sizeB; j++) {
        for (int k = 0; k < sizeA; k++) {
            if (b[j] != a[k]) {
                count++;
            }
        }

        //если счётчику стал равен размеру массива a(то есть число из массива
b не совпало с элементами массива a)
        //записываем этот элемент в массив c
        if (count == sizeA) {
            c[n] = b[j];
            n++;
        }

        count = 0;
    }

    //записываем размер массива c
```

```

        *sizeC = n;
    }

void intersectionMany(int const * a, int sizeA, int const * b, int sizeB, int
*c, int *sizeC) {
    int n = 0;

    //находим элементы в массиве a и b, которые совпадают
    for (int i = 0; i < sizeA; i++) {
        for (int j = 0; j < sizeB; j++) {
            if (a[i] == b[j]) {
                //записываем эти элементы в массив c
                c[n++] = a[i];
            }
        }
    }

    //сохраняем размер массива c
    *sizeC = n;
}

void differenceSet(int const *a, int sizeA, int const *b, int sizeB, int *c,
int *sizeC) {
    int n = 0;
    int i = 0;
    int j = 0;
    int count = 0;

    //пока индексы не равны размерам массива, ведём запись
    while (i != sizeA) {
        //если элемент из массива a меньше элемента из массива b записываем
его, так как массивы отсортированы
        //иначе если элемент из массива a больше элемента из массива b идём
дальше по массиву b, счётчик увеличиваем на 1
        //иначе идём дальше по массиву a, так как элементы равны
        if (a[i] < b[j]) {
            c[n++] = a[i];
            i++;
        } else if (a[i] > b[j]){
            j++;
            count++;
        } else {
            i++;
        }
        //если счётчик стал равен размеру массива b(то есть элемента из
массива a нет в массиве b)
        //тогда мы записываем элемент a в массив c, идём дальше по массиву a
на новый элемент, счётчик и индекс
        //массива b обнуляем
        if (count == sizeB) {
            c[n++] = a[i];
            i++;
            count = 0;
            j = 0;
        }
    }

    //записываем размер массива c
    *sizeC = n;
}

int main() {
    int sizeA = 4;
    int sizeB = 4;

```

```

int sizeC = 4;
int sizeRes1 = 0;
int sizeRes2 = 0;
int sizeRes3 = 0;
int sizeLast1 = 0;

int a[sizeA];
int b[sizeB];
int c[sizeC];
inputArray(a, sizeA);
inputArray(b, sizeB);
inputArray(c, sizeC);

int res1[sizeA + sizeB];
int res2[sizeA + sizeC];
int res3[sizeB + sizeC];
int lastResult1[sizeRes1 + sizeRes2];

intersectionMany(a, sizeA, b, sizeB, res1, &sizeRes1);

differenceSet(a, sizeA, c, sizeC, res2, &sizeRes2);

differenceSet(b, sizeB, c, sizeC, res3, &sizeRes3);
insertionSort(res3, sizeRes3);
outputArray(res3, sizeRes3);

unionSets(res1, sizeRes1, res2, sizeRes2, lastResult1, &sizeLast1);

insertionSort(lastResult1, sizeLast1);
outputArray(lastResult1, sizeLast1);
}

```

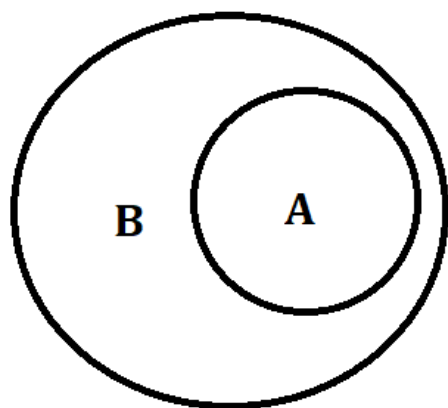
Ответ {5, 6, 1, 3, 8}

4. Вывод

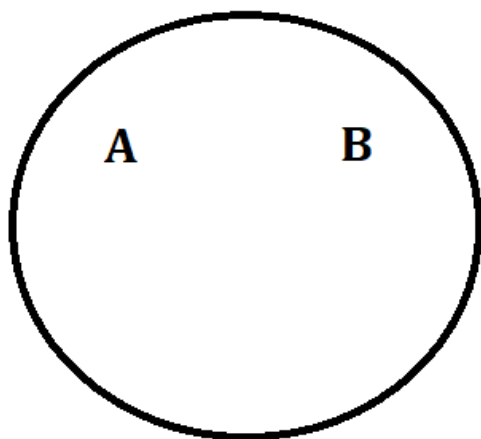
Я изучил и научился использовать алгебру подмножеств, изучил различные способы представления множеств в памяти ЭВМ, научился программно реализовывать операции над множествами и выражения в алгебре подмножеств.

5.

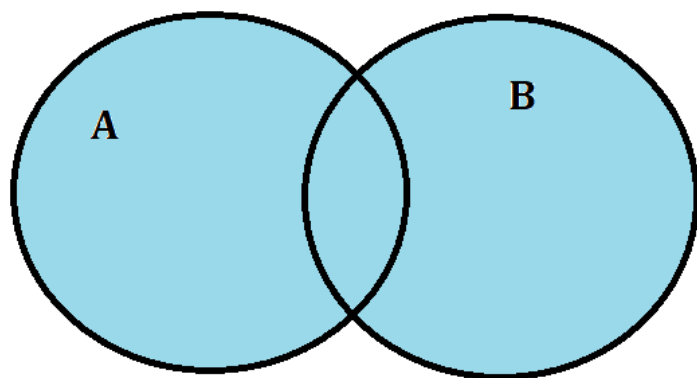
Включение



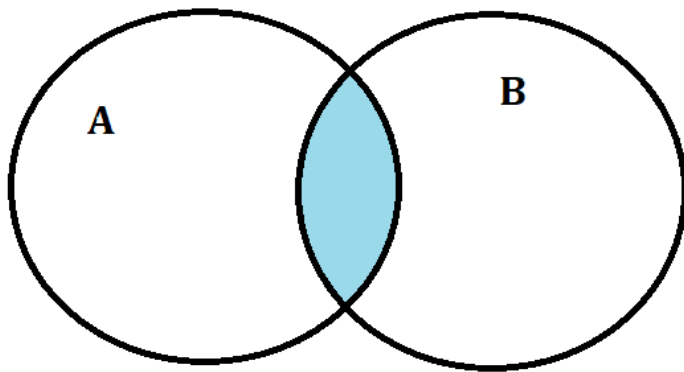
Равенство



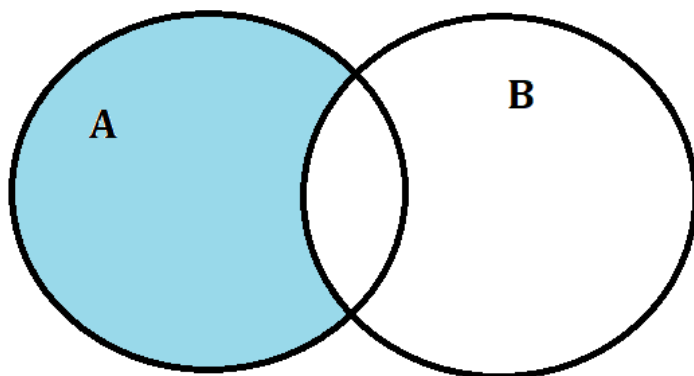
Объединение



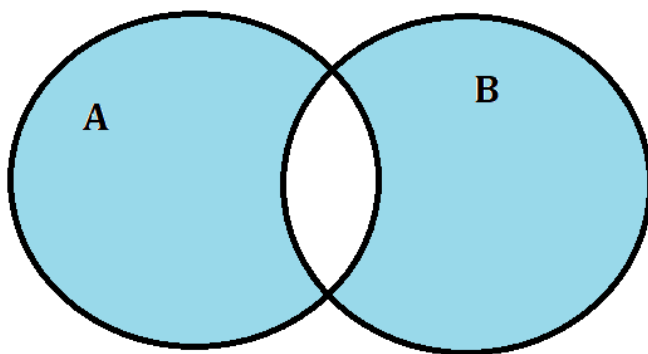
Пересечение



Разность



Симметрическая разность



6.

Множество называется бесконечным, если оно состоит из бесконечного числа элементов. Таковы, например, множество всех натуральных чисел, множество точек окружности, множество прямых, проходящих через точку плоскости

Например

а) $A = \{x \mid x \in \mathbb{N}\}$

б) а) $A = \{x \mid x \in \mathbb{R}\}$